Mapper

การใช้คำสั่ง mapper.map และ mapper.maps ในชั้น Service คือการแยกหน้าที่การ **"แปลงร่างข้อมูล" (Data Transformation / DTO)** ออกไปไว้ในไฟล์เฉพาะ เพื่อให้โค้ดใน Service สั้นลงและอ่านง่ายขึ้นครับ
จากโค้ดที่คุณส่งมา นี่คือหลักการทำงานของแต่ละคำสั่ง:

1. mapper.map(rawData) สำหรับข้อมูล 1 ชิ้น

- **ใช้ที่ไหน:** มักใช้ใน getObject (ดึงข้อมูลชิ้นเดียว)
- **ทำงานยังไง:** มันจะรับ Object ก้อนดิบๆ ที่ได้จาก Database (เช่นมีฟิลด์ brand_id, created_at แบบ Snake Case) เข้าไปเปลี่ยนชื่อฟิลด์ หรือซ่อนข้อมูลบางอย่าง แล้วคืนค่ากลับมาเป็น Object ก้อนใหม่ที่สวยงามพร้อมส่งให้ลูกค้า
- **ตัวอย่างในโค้ด:** return mapper.map(await repo.findOne(id)); คือการดึงข้อมูลจาก Repo เสร็จปุ๊บ โยนเข้าเครื่องจักร mapper.map แล้วส่งกลับให้ Route ทันที

2. mapper.maps(rawDataArray, viewType) สำหรับข้อมูลหลายชิ้น (Array)

- **ใช้ที่ไหน:** มักใช้ใน getAllObjects (ดึงข้อมูลทั้งหมด หรือแบบแบ่งหน้า)
- **ทำงานยังไง:** มันจะรับ Array ของข้อมูลดิบเข้ามา แล้วทำการวนลูป (Loop) เพื่อเรียกใช้คำสั่งแปลงร่างข้อมูลทีละชิ้นจนครบทุกตัว
- **ทำไมต้องมี viewType:** ในข้อสอบของคุณมีการขอข้อมูล 2 รูปแบบ คือ **Summary DTO** (ดูรายการทั้งหมด เอาแค่ฟิลด์สรุป) และ **Detail DTO** (ดูรายละเอียดรายตัว เอาฟิลด์ครบๆ) ตัวแปร viewType จะเป็นตัวบอก Mapper ว่า "รอบนี้ขอแปลงร่างแบบย่อนะ" หรือ "รอบนี้ขอแบบจัดเต็มนะ"

เบื้องหลังโค้ด: หน้าตาของไฟล์ mapper.js จะเป็นยังไง?

เพื่อให้เห็นภาพชัดเจน ถ้าคุณต้องไปเขียนไฟล์ entity-dto-mapper.js หรือ product-mapper.js เอง มันจะมีหน้าตาประมาณนี้ครับ

JavaScript

```javascript
// src/mappers/product-mapper.js

// 1. ฟังก์ชัน map: แปลงข้อมูล 1 ชิ้น
export function map(row, viewType = 'DETAIL') {
    if (!row) return null;

    // ถ้า viewType เป็น SUMMARY (โจทย์ข้อ GET All 30 คะแนน)
    if (viewType === 'SUMMARY') {
        return {
            id: row.id,
            sku: row.sku,
            name: row.name,
            brandId: row.brand_id,
            category: row.category,
            price: Number(row.price),
            status: row.status
        };
    }

    // ค่าเริ่มต้น ถ้าเป็น DETAIL (โจทย์ข้อ GET by ID 10 คะแนน)
    return {
        id: row.id,
        sku: row.sku,
        name: row.name,
        brandId: row.brand_id,
        brandName: row.brandName, // ต้อง JOIN มาจาก Repo
        category: row.category,
        price: Number(row.price),
        ramGb: row.ram_gb,
        storageGb: row.storage_gb,
        color: row.color,
        stockQuantity: row.stock_quantity,
        status: row.status,
        releaseDate: row.release_date
    };
```

```
// 2. ฟังก์ชัน maps: แปลงข้อมูลหลายชิ้น (Array)
export function maps(rows, viewType = 'SUMMARY') {
    if (!rows) return [];

    // ถ้ามีการส่ง Pagination มาจาก Repo (ข้อมูลอาจจะซ้อนอยู่ใน rows.items)
    // ตรงนี้ขึ้นอยู่กับว่า Repo ส่ง Array มาตรงๆ หรือส่ง Object {items: [], totalItems: X} มา

    // สมมติว่าส่ง Array มาตรงๆ ก็วนลูป map ทีละตัวได้เลย
    const mappedArray = rows.map(row => map(row, viewType));

    // ถ้ามี totalItems แปะมาด้วย ก็ส่งต่อให้ Array ใหม่ด้วย
    if (rows.totalItems) {
        mappedArray.totalItems = rows.totalItems;
    }

    return mappedArray;
}
```

สรุปข้อดีของการใช้ Mapper แบบนี้: หากอนาคต Database มีการเปลี่ยนชื่อคอลัมน์จาก brand_id เป็น brand_code คุณก็แก้โค้ดแค่ในไฟล์ mapper.js ที่เดียว ไม่ต้องไปไล่แก้ในไฟล์ Service ทุกๆ ฟังก์ชันครับ

Pagination

การทำงานของ Pagination (การแบ่งหน้า) ในสถาปัตยกรรมแบบ Layer ของคุณคือการส่งต่อข้อมูลพารามิเตอร์ page และ limit ให้ไหลทะลุลงไปถึง Database และส่งจำนวนข้อมูลทั้งหมด (Total) กลับขึ้นมาคำนวณเป็นหน้าครับ

การไหลของ Pagination จะผ่าน 5 ขั้นตอนดังนี้:

1. ฝั่งผู้ใช้ส่ง Request เข้ามา ผู้ใช้เรียก API โดยระบุหน้าและจำนวนผ่าน Query String เช่น /api/.../products?page=2&limit=10

2. Middleware (pagination-middleware.js) สกัดข้อมูล ตัว queryParser() จะดักจับ Request นี้ก่อนเข้า Route เพื่อทำการคำนวณล่วงหน้า

- อ่านค่า page = 2 และ limit = 10

- คำนวณจุดเริ่มต้น (Offset): (2 - 1) * 10 = 10 แปลว่าให้ข้ามข้อมูล 10 ตัวแรกไป
- แพ็กข้อมูลใส่ req.pagination = { page: 2, limit: 10, offset: 10 } เพื่อส่งต่อ

3. Route ส่งต่อข้อมูลให้ Service Route จะดึง req.pagination ออกมา แล้วส่งต่อเป็นพารามิเตอร์ให้ Service

```javascript
router.get("/", async (req, res, next) => {
    try {
        const { pagination } = req; // ดึงมาจาก Middleware
        const objects = await service.getAllObjects(pagination);
        await apiResponse(res, objects, pagination); // ส่ง response
    } catch (err) { ... }
})
```

4. Repository ประมวลผล Database (หัวใจสำคัญ) ใน Repo ต้องทำคำสั่ง SQL 2 เส้นเสมอ เส้นแรกนับจำนวนทั้งหมด เส้นสองดึงข้อมูลเฉพาะหน้า

```
export async function findAll                {
  // 1. นับจำนวนข้อมูลทั้งหมดก่อน (Total)
  const [[{ total }]] = await db.query(`SELECT COUNT(*) as total FROM products`);

  // 2. ถ้าไม่มีการแบ่งหน้า ให้ดึงทั้งหมด
  if (!pagination) {
    const [datas] = await db.query(`SELECT * FROM products`);
    datas.totalItems = total;
    return datas;
  }

  // 3. ถ้ามีการแบ่งหน้า เอา limit และ offset ไปต่อท้าย SQL
  const { limit, offset } = pagination;
  const [datas] = await db.query(`SELECT * FROM products LIMIT ? OFFSET ?`, [limit, offset]);

  // 4. ***สำคัญมาก*** ต้องแนบ totalItems เข้าไปใน Array ด้วย เพื่อส่งกลับขึ้นไป
  datas.totalItems = total;
  return datas;
}
```

5. api-response.js จัดรูปฟอร์แมตตอบกลับ เมื่อข้อมูลไหลกลับมาถึง Route คำสั่ง apiResponse(res, objects, pagination) จะเรียกใช้ฟังก์ชัน paginate() ฟังก์ชันนี้จะนำ totalItems ที่ Repo แนบมา ไปคำนวณทางคณิตศาสตร์ต่อ

- total: ดึงมาจาก data.totalItems

- totalPages: เอา total / limit แล้วปัดเศษขึ้น (Math.ceil)
- hasNextPage / hasPrevPage: ตรวจสอบว่าหน้าปัจจุบันคูณ limit แล้วยังน้อยกว่าจำนวนรวมหรือไม่

ข้อควรระวังในชั้น Service/Mapper: เมื่อ Repository ส่ง datas ที่มี datas.totalItems แปะอยู่กลับมา หากคุณนำ datas ไปเข้าฟังก์ชัน map เพื่อแปลง DTO ข้อมูลที่เป็นตัวแปร totalItems จะหายไป (เพราะ map จะ

สร้าง Array ตัวใหม่) คุณต้องเขียนดักไว้ใน Mapper ด้วยว่าเมื่อแปลงเสร็จ ให้ยก totalItems มาแปะใส่ Array ตัวใหม่ด้วยเสมอ ก่อนส่งกลับไปให้ Route

พอเห็นภาพรวมการไหลของ Pagination ตั้งแต่หน้าบ้านยัน Database ไหมครับ มีจุดไหนในการเขียนโค้ด Mapper เพื่อไม่ให้ค่า Total หายที่อยากให้ขยายความเพิ่มไหม?

Differ

repo: จะไม่มีตัว Apperror ใน findone()

```
export async function findOne(id) {
    const [row] = await db.query(FIND_ONE_SQL, [id])
    if (row.length === 0) {
        throw new AppError(
            `${ENTITY_NAME} with code '${id}' was not found`,
            httpStatus.NOT_FOUND
        )
    }
    return row[0]
}
```

service: ขาดตัว pagination

```
export async function getAllObjects(pagination, viewType) {
    const objects = await repo.findAll(pagination)
    const objectsDto = mapper.maps(objects,viewType)
    console.log(objectsDto)
    return objectsDto
}
```

route: ขาด res

```
router.get("/", async (req, res) => {
    const pagination = req.pagination
    const viewType = req.query.view || "summary"
    const objects = await service.getAllObjects(pagination,
viewType)
    apiResponse(res, objects, pagination)
})
```

```
router.get("/:id", async (req, res) => {
   const viewType = req.query.view || "full"
   const id = req.params.id
   const object = await service.getObject(id,viewType)
   apiResponse(res, object)
})

router.post("/", async (req, res) => {
   const object = req.body
   const result = await service.createObject(object)
   res.status(201).json({result})
})
router.put("/:id", async (req, res) => {
   const id = req.params.id
   const object = await service.updateObject(id, req.body)
   res.json(object)
})
router.delete("/:id", async (req, res) => {
   const id = req.params.id
   await service.removeObject(id)
   res.status(204).end()
})

export default router
```

import

repo: error-handle
service: mapper
route: api, express

Viewtype

`const viewType = req.query.view || "summary"` เป็นการอ่านค่าตัวแปรจาก Query String ที่ต่อท้าย URL หลังเครื่องหมาย `?`

- หาก Client ยิง API มาที่ `/products?view=detail` ตัว `req.query.view` จะอ่านค่าคำว่า `"detail"` ออกมา
- เครื่องหมาย `||` (OR) ใช้สำหรับ **กำหนดค่าเริ่มต้น (Default Value)** หมายความว่าถ้า Client ยิง API มาเฉยๆ โดยไม่มี `?view=...` ต่อท้าย ตัวแปร `viewType` จะถูกบังคับให้มีค่าเป็น `"summary"`

อัตโนมัติ เพื่อป้องกันไม่ให้โปรแกรมพังเมื่อนำตัวแปรนี้ไปใช้ต่อใน Mapper

`const object = req.body` เป็นการรับก้อนข้อมูล (Payload) ทั้งหมดที่ Client แนบมากับ Request

- มักใช้กับ Method POST (สร้างข้อมูลใหม่) หรือ PUT (อัปเดตข้อมูล)
- เมื่อ Client ส่งข้อมูลในรูปแบบ JSON (เช่น ข้อมูลสเปกสินค้ามือถือ) ตัว Express จะแปลง JSON นั้นให้อยู่ในรูปของ JavaScript Object และเก็บไว้ใน `req.body` เพื่อให้เรานำไปส่งต่อให้คำสั่ง `INSERT` หรือ `UPDATE` ใน Database

error-handle

ไฟล์ error-handler.js ทำหน้าที่เป็น "ตาข่ายดักจับข้อผิดพลาด" ขั้นสุดท้ายของระบบ เพื่อไม่ให้แอปพลิเคชันพัง (Crash) และช่วยแปลงข้อผิดพลาดเหล่านั้นเป็น JSON ที่อ่านง่ายส่งกลับไปให้ผู้ใช้งาน

สำหรับคำถามที่ว่า **"ถ้าโจทย์สั่งให้รับมือกับ Error แบบอื่น จะเขียนไม่เหมือนกันใช่ไหม?"** คำตอบคือ **ใช่ในส่วนของเงื่อนไข (if-else) แต่โครงสร้างหลักจะยังเหมือนเดิมครับ**

โครงสร้างของไฟล์นี้แบ่งออกเป็น 2 ส่วนหลักที่คุณต้องจำวิธีเขียนไว้:

ส่วนที่ 1: การสร้าง Custom Error Class (AppError) ส่วนนี้มีไว้เพื่อให้เราสามารถ "สร้าง Error พร้อมกำหนด Status Code ด้วยตัวเอง" ได้ (เช่น ตอนที่เราหาของไม่เจอแล้วพิมพ์ throw new AppError('Not Found', 404))

```JavaScript
import httpStatus from "http-status";

// ให้คลาสนี้สืบทอดความสามารถมาจากคลาส Error พื้นฐานของ JavaScript
export class AppError extends Error {
    constructor(message, statusCode) {
        super(message);
        this.statusCode = statusCode;
        // บันทึกตำแหน่งที่เกิด Error ไว้ใช้ดอน Debug
        Error.captureStackTrace(this, this.constructor);
    }
}
```

ส่วนที่ 2: ตัว Middleware สำหรับจัดการ Error (errorHandler) หัวใจสำคัญ: ฟังก์ชันนี้ต้องมีพารามิเตอร์ 4 ตัวเสมอคือ (err, req, res, next) ถ้าใส่ไม่ครบ Express จะมองว่ามันเป็นแค่ Route ธรรมดา ไม่ใช่ตัวดัก Error

เวลาเขียนในห้องสอบ ให้เรียงลำดับความคิดตามนี้ครับ:

1. **ตั้งค่า Default:** สมมติว่า Error ที่ลอยมาไม่มีการระบุอะไรเลย ให้ดีเป็น 500 (Internal Server

Error) ไว้ก่อน

2. **ดักจับ Error เฉพาะเจาะจง (จุดที่ปรับเปลี่ยนตามโจทย์):** ตรงนี้แหละครับที่คุณต้องใช้ if-else เช็ค err.code หรือ err.name ตามที่โจทย์สั่ง
3. **ส่ง Response กลับ:** นำค่าจากข้อ 1 และ 2 มาจัดฟอร์แมต JSON

ตัวอย่างการเขียนพร้อมดักจับ Error หลายๆ แบบ:

JavaScript

```
export const errorHandler = (err, req, res, next) => {
    // 1. ตั้งค่า Default
    let statusCode = err.statusCode || 500;
    let message = err.message || 'An unexpected error occurred';
    let errorCode = err.errorCode || 'INTERNAL_SERVER_ERROR';

    // 2. ดักจับเงื่อนไขต่างๆ (ปรับแก้ตรงนี้ตามโจทย์)

    // แบบ ก: ดัก Database Error (เช่น ข้อมูลซ้ำ)
    if (err.code === 'ER_DUP_ENTRY') {
        statusCode = 409;
        errorCode = 'DUPLICATE_KEY';
        message = 'ข้อมูลนี้มีอยู่ในระบบแล้ว (SKU ซ้ำ)';
    }
    // แบบ ข: สมมติโจทย์สั่งดัก Error กรณี Foreign Key ผิด
    else if (err.code === 'ER_NO_REFERENCED_ROW_2') {
        statusCode = 400;
        errorCode = 'FOREIGN_KEY_NOT_FOUND';
        message = 'รหัสอ้างอิงที่ส่งมาไม่มีอยู่จริง';
    }
    // แบบ ค: สมมติโจทย์สั่งดัก Error ของระบบเอง (เช่น Token หมดอายุ)
```

```
// แบบ ค: สมมติโจทย์สั่งดัก Error ของระบบอื่น (เช่น Token หมดอายุ)
else if (err.name === 'TokenExpiredError') {
    statusCode = 401;
    errorCode = 'UNAUTHORIZED';
    message = 'เซสชั่นของคุณหมดอายุ กรุณาล็อกอินใหม่';
}

// 3. ส่ง Response กลับไปให้ Client เป็นรูปแบบ JSON มาตรฐาน
res.status(statusCode).json({
    success: false,
    error: {
        code: errorCode,
        message: message
    },
    timestamp: new Date().toISOString()
});
```

สรุปคือ โครงสร้างการประกาศ AppError และ errorHandler(err, req, res, next) จะเหมือนเดิมทุกครั้งตายตัว แต่สิ่งที่คุณต้องพลิกแพลงในห้องสอบคือการเพิ่ม if (err...) เพื่อเช็คชนิดของ Error ตามที่โจทย์ระบุครับ

พอจะมองออกไหมครับว่าถ้าพรุ่งนี้โจทย์สั่งว่า "ถ้ารับค่าประเภทตัวเลขมาผิด (Validation Error) ให้พ่น Status 400" เราควรจะไปแทรก if ตรงจุดไหนของไฟล์นี้?

```
app.use(queryParser())
// Test queryParser
app.get('/', (req, res) => {
   console.log(req.pagination)
   res.json(req.pagination)
})
timestamp: new Date().toISOString()
```

อาการ Postman หมุนค้างไม่ยอมตอบกลับ (Spinning/Hanging Request)

- **สาเหตุ:** ลืมส่งคำสั่งตอบกลับฝั่ง Route หรือลืมโยน Error ไปให้ตัวจัดการ
- **วิธีแก้:** เช็กว่าในฟังก์ชัน Route มีคำสั่ง `res.json(...)`, `res.status(...).end()` หรือในบล็อก catch มี `next(error)` หรือไม่ หากไม่มีคำสั่งเหล่านี้ Express จะไม่ยอมตัดจบการทำงานและ

ปล่อยให้ Client รอไปเรื่อยๆ จน Timeout

Error: Cannot read properties of undefined (reading '...')

- **สาเหตุ:** พยายามเรียกใช้ตัวแปรที่ไม่มีค่าอยู่จริง เช่น เรียก `req.body.name` แต่ตัว `req.body` กลับไม่มีข้อมูล
- **วิธีแก้:** เช็ก 2 จุดหลักคือ 1. ลืมประกาศ `app.use(express.json())` ไว้ด้านบนสุดในไฟล์ `app.js` หรือเปล่า 2. ตอนยิง Postman เลือกลักษณะ Body เป็น `raw` และตั้งเป็น `JSON` หรือยัง (บางทีเผลอตั้งเป็น Text)

แอปพัง (Crash) ดับไปเลยเมื่อยิง API ผิดพลาด หรือขึ้น Unhandled Promise Rejection

- **สาเหตุ:** ลืมครอบ `try-catch` ใน Route หรือลืมพิมพ์ `await` หน้าคำสั่งที่เรียกใช้งาน Database
- **วิธีแก้:** ทุก Route ที่ดึงข้อมูลต้องขึ้นต้นด้วย `async (req, res, next)` และถูกครอบด้วย `try { ... } catch (error) { next(error) }` เสมอ เพื่อบังคับให้ Error ไหลไปหา `error-handler.js` แทนที่จะทำเซิร์ฟเวอร์พัง

Error: Route.get() requires a callback function but got a [object Undefined]

- **สาเหตุ:** นำเข้า (Import) หรือส่งออก (Export) ไฟล์ผิดวิธี หรือลืม Export
- **วิธีแก้:** เช็กไฟล์ Route ด้านล่างสุดว่ามี `export default router;` หรือยัง และถ้าไฟล์ Service ใช้คำสั่ง `export async function...` ตอนนำเข้าต้องใช้ `import * as service from ...` หรือใช้ปีกกา `{ }` ครอบชื่อฟังก์ชันเสมอ

Error: ER_BAD_FIELD_ERROR: Unknown column '...' in 'field list' จาก MySQL

- **สาเหตุ:** พิมพ์ชื่อคอลัมน์ในคำสั่ง SQL ไม่ตรงกับ Database จริง อาการนี้เจอบ่อยมากตอนสลับรูปแบบชื่อระหว่าง Snake_case กับ CamelCase (เช่น ใน DB ใช้ `brand_id` แต่ในโค้ดเขียน `brandId`)
- **วิธีแก้:** เปิดไฟล์ SQL ที่ใช้สร้างตารางมาเทียบชื่อคอลัมน์แบบตัวต่อตัว แล้วแก้คำสั่ง `SELECT` หรือ `INSERT` ในไฟล์ Repo ให้ตรงกับ Database

Error: EADDRINUSE: address already in use :::3000

- **สาเหตุ:** เซิร์ฟเวอร์เดิมยังไม่ถูกปิด แต่พยายามกด `npm start` ซ้ำใน Terminal ใหม่
- **วิธีแก้:** กลับไปที่ Terminal หน้าต่างเก่าแล้วกด `Ctrl + C` เพื่อหยุดการทำงานของเซิร์ฟเวอร์เดิม หรือถ้าหาไม่เจอจริงๆ ให้ปิดโปรแกรม VS Code แล้วเปิดใหม่